

Serial-Links Design – Cadence Tools

Lab 07

PLL

Intended Learning Objectives

This lab is divided into two parts:

- In Part 1 you will
 - Learn how to design Type II PLL circuit parameters (R_z , C_z , C_p) for a stable PLL.
 - Run STB analysis for PLL linear model and measure its stability criteria.
 - See some trade-offs in PLL stability, and how circuit components affects it.
- In Part 2 you will
 - See PLL sub-blocks VerilogA models.
 - Learn how to test each PLL sub-block and see its performance metrics.
 - Run transient analysis for PLL, and see start-up & phase-lock regions.
- In Part 3 you will
 - See real circuit implementation of PLL different sub-blocks.
 - Design integer-N frequency divider, using smller-N dividers.
 - Test PLL circuits and see their performance metrics.
 - Run transient analysis for PLL with real circuitry, and see start-up & phase-lock regions.

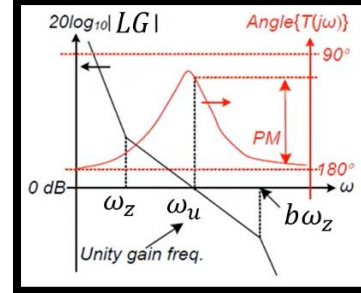
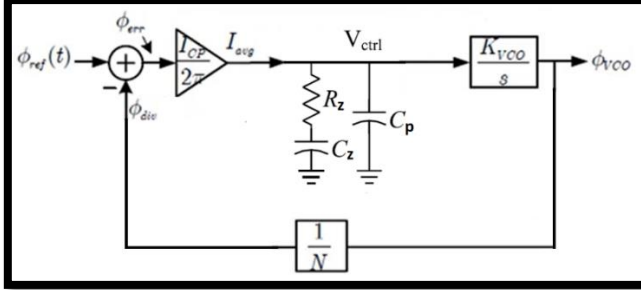
In this lab, we will design a PLL with the following specs:

	Spec
V_{DD}	1 V
f_{in}	100 MHz
f_{out}	5 GHz
$f_{BW,CL}$	$0.05 f_{in} = 5 \text{ MHz}$
Phase Margin	62°
K_{VCO}	$2\pi * \frac{20}{3} \text{ G rad/V}$

PART 1: PLL Behavioral Model

1. Calculating Circuit Parameters

We studied in the lecture, the LTI model of Type II PLL. Here is a small recap:



$$A_{CL} = \frac{N \left(1 + \frac{s}{\omega_z}\right)}{1 + \frac{s}{\omega_z} + \frac{s^2}{\omega_z \omega_u} + \frac{s^3}{b \omega_z^2 \omega_u}}, \quad \omega_p = b \omega_z$$

$$LG = \omega_u \omega_z \left(\frac{1 + \frac{s}{\omega_z}}{s^2 \left(1 + \frac{s}{b \omega_z}\right)} \right), \quad PM = \tan^{-1} \left(\frac{\omega_u}{\omega_z} \right) - \tan^{-1} \left(\frac{\omega_u}{b \omega_z} \right) \rightarrow (1)$$

$$\omega_u = \frac{b-1}{b} * \frac{I_{CP} K_{VCO} R_z}{2\pi N} \rightarrow (2), \quad \omega_z = \frac{1}{R_z C_z} \rightarrow (3), \quad b = 1 + \frac{C_z}{C_p} \rightarrow (4)$$

Given $PM, \omega_{BW,CL}$,

Assume: $\omega_{BW,CL} = \omega_u = \sqrt{\omega_z \cdot b \omega_z}$

- 1.1. Calculate $N, b, C_z/C_p$.
- 1.2. Calculate $f_z, f_u, f_p = b f_z$.
- 1.3. Let $I_{CP} = 10\mu A$, get R_z .
- 1.4. Calculate C_z & C_p .

2. Creating Testbench

- We want to generate this PLL TestBench:

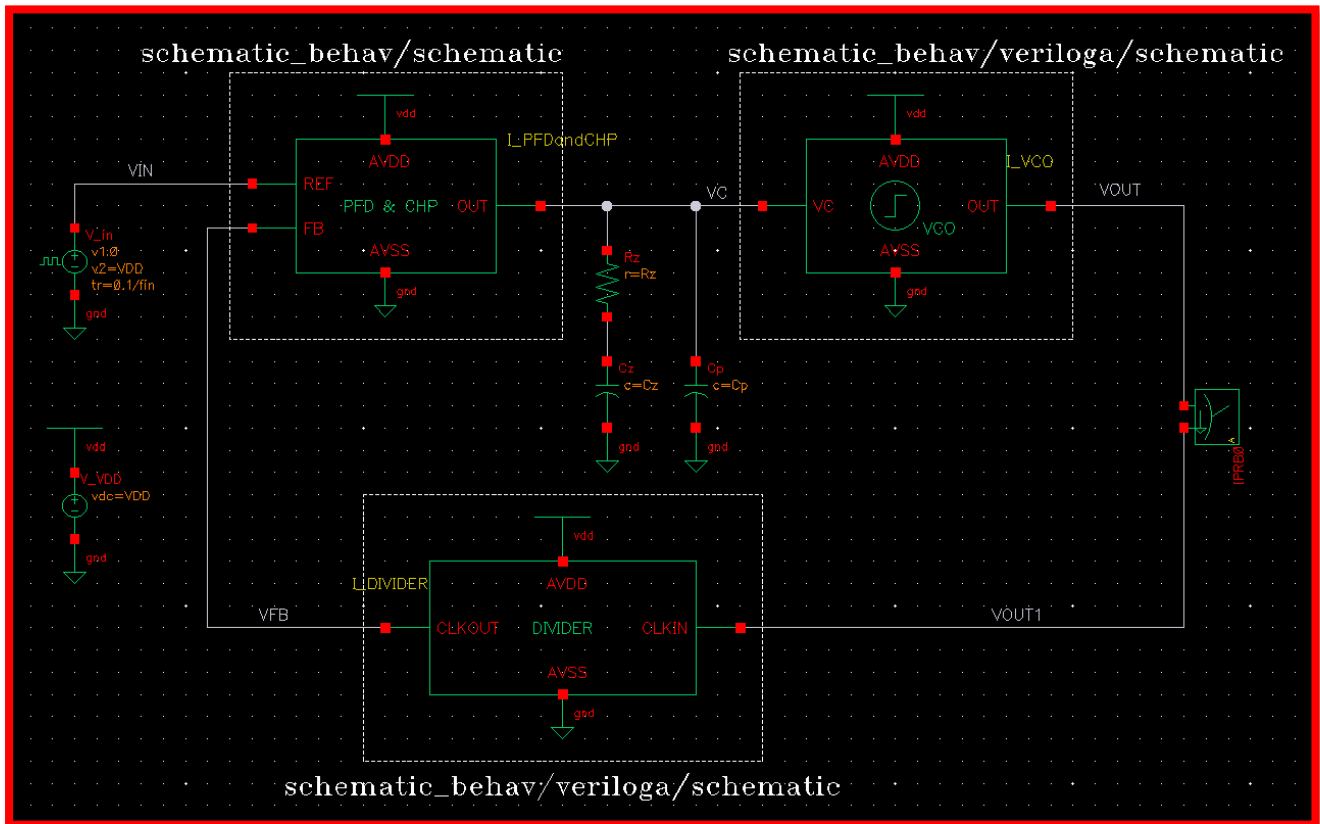


Figure 1: Main PLL TB

- Create three cells for “PFDandCHP”, “VCO” & “DIVIDER”. Cells are of type “schematic” and name the view: “schematic_behav”.
- Generate symbols from schematics.
- Draw TB schematic as in Figure 1: Main PLL TB.

Voltage 1	0 V
Voltage 2	VDD V
Period	1/fin s
Delay time	
Rise time	0.1/fin s
Fall time	0.1/fin s
Pulse width	0.4/fin s

Make input parameters as follows:

- Generate config view for this TB schematic, and generate ADEXL to this config view. Refer to the following video from Master Micro Channel, for more config view setup: https://www.youtube.com/watch?v=LQqCtnEzfQ0&list=PLMSBalys69yza1LVr3JNc6_pnYW4CvIEb&index=5, from 13:00 to 16:30.

PFDandCHP:

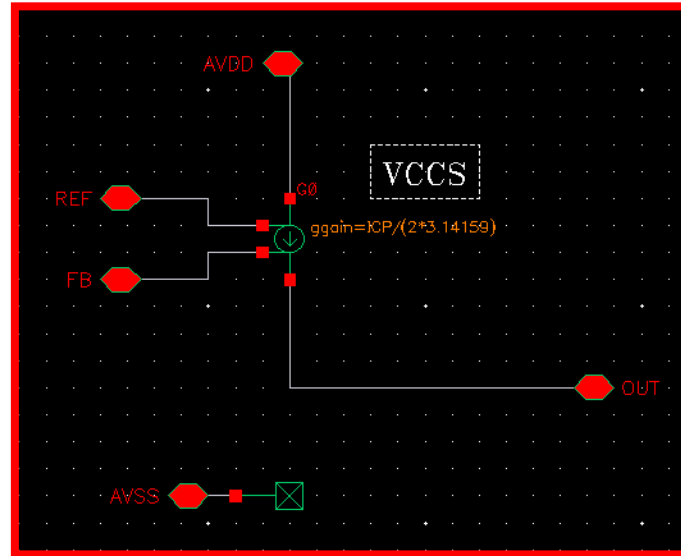


Figure 2: PFD&CHP Behavioral Model

VCO:

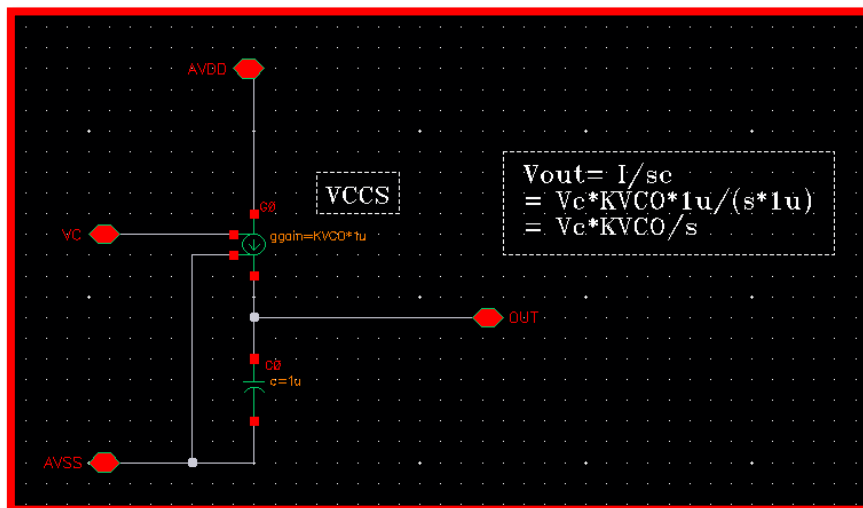


Figure 3: VCO Behavioral Model

DIVIDER:

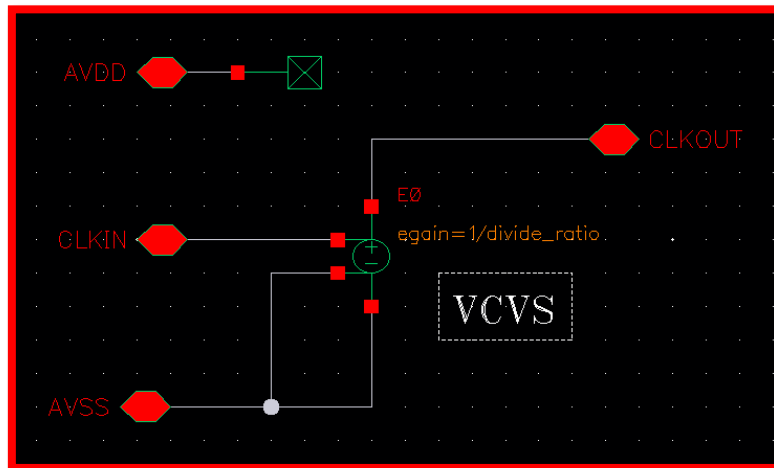


Figure 4: Frequency Divider Behavioral Model

3. Stability (AC) Simulation

- 3.1. Run “stb” analysis, and plot LoopGain (mag in dB & phase).
- 3.2. Measure f_z , f_u , f_p , $PhaseMargin$. Comment on your results.
- 3.3. Sweep R_z from 10k: 40k with step 5k. Plot the LG (mag in dB & phase). Plot UGF & PM vs R_z . Comment on your results.
- 3.4. Return R_z to the old designed value. Sweep I_{CP} from 2uA: 22uA with 4uA step. Plot the LG (mag in dB & phase). Plot UGF & PM vs I_{CP} . Comment on your results.

Part 2: PLL Verilog-a Model

1. TestBench Setup

- Create a new schematic view for “PFDandCHP” block.

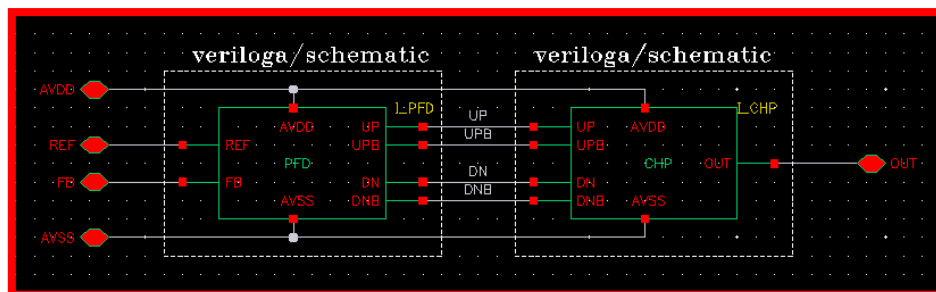


Figure 5: PFDandCHP schematic

- Create VerilogA cell for “PFD”, “CHP”, “VCO”, “DIVIDER”. VerilogA Model can be shown below.

PFD:

```
// VerilogA for Analog_Course, pll_PFD, veriloga
`include "constants.vams"
`include "disciplines.vams"

// REF: Reference signal
// FB: Feedback signal
// UP: Up signal (FB late)
// DN: Down signal (FB early)

module serdes_iti_lab7_pll_PFD( FB, REF, UP, UPB, DN, DNB, AVDD, AVSS);

inout FB, REF, UP, UPB, DN, DNB, AVDD, AVSS;
electrical FB, REF, UP, UPB, DN, DNB, AVDD, AVSS;

    // VDD and threshold voltage for digital signals
    parameter real VDD = 1.2;
    parameter real thresh = 0.6;
    // rise/fall/delay times of PFD output
    parameter real trise = 10p, tfall = 10p, td = 0;

    // Internal UP and DN signals
    // *** add line here ***
    integer UP_i = 0, DN_i = 0;

    analog begin

        // Check the FB signal state when REF arrives
        // *** add line here ***
        @(cross(V(REF)-thresh,1))
            if(V(FB) < thresh)
                // *** add line here ***
                UP_i = 1;
            else begin
                UP_i = 0;
                DN_i = 0;
            end

        // Check the REF signal state when FB arrives
        @(cross(V(FB)-thresh,1))
            // *** add line here ***
            if(V(REF) < thresh)
                DN_i = 1;
            else begin
                // *** add line here ***
                // *** add line here ***
                UP_i = 0;
                DN_i = 0;
            end

        V(UP) <+ transition( UP_i*VDD,td,trise,tfall);
        // *** add line here ***
        V(DN) <+ transition( DN_i*VDD,td,trise,tfall);

        V(UPB) <+ transition( !UP_i *VDD,td,trise,tfall);
        V(DNB) <+ transition( !DN_i*VDD,td,trise,tfall);

    end

endmodule
```

Figure 6: PFD VerilogA Model

CHP:

```
// VerilogA for Analog_Course, pll_CHP, veriloga
// AMS PLL Project: Charge Pump (CHP)

`include "constants.vams"
`include "disciplines.vams"

// UP: Up signal
// DN: Dn signal
// IOUT: CHP current output
// Since the output is current, IOUT cannot be left unconnected (o.c.) in the testbench
module serdes_iti_lab7_pll_CHP(UP,UPB,DN,DNB,OUT, AVDD, AVSS);
    inout UP, UPB, DN, DNB, OUT, AVDD, AVSS;
    electrical UP, UPB, DN, DNB, OUT, AVDD, AVSS;

    // ichp: CHP current
    parameter real ichp = 10u from [0:inf];
    // Threshold voltage for digital signals
    parameter real thresh=0.6;
    // rise/fall/delay times of CHP output
    parameter real trise=10p, tfall=10p, td=0;

    // Internal variable for CHP output current
    real OUT_i = 0;

    analog begin

        // Generate events at UP and DN transitions
        @( cross(V(UP)-thresh,0) or cross(V(DN)-thresh,0) );
        // *** add line here ***
        // *** add line here ***

        if ((V(UP) > thresh) && (V(DN) < thresh))
            OUT_i = -1; // Output Current (Out of the symbol)
        // *** add line here ***
        else if ((V(UP) < thresh) && (V(DN) > thresh))
            OUT_i = 1; // Input Current (to the symbol)
        else
            // *** add line here ***
            OUT_i = 0;

        I(OUT) <+ transition(OUT_i*ichp ,td,trise,tfall);

    end
endmodule
```

Figure 7: CHP VerilogA Model

VCO:

```
// VerilogA for Analog_Course, pll_VCO, veriloga

`include "constants.vams"
`include "disciplines.vams"

module serdes_iti_lab7_pll_VCO(VC, OUT, AVDD, AVSS);
inout VC, OUT, AVDD, AVSS;
electrical VC, OUT, AVDD, AVSS;

    parameter real VDD = 1.2;
    parameter real Vmin=0.2;
    parameter real Vmax=1 from (Vmin:inf);
    parameter real Fmin=0.5G from (0:inf);
    parameter real Fmax=1.5G from (Fmin:inf);
    parameter real trise=10p, tfall=10p, td=0;

    // *** add line here ***
    real VOUT_i, phase, freq, sine;

    analog begin

        // compute the freq from the input voltage
        freq=((V(VC) - Vmin)*(Fmax - Fmin) / (Vmax - Vmin)) + Fmin;
        // bound the frequency
        // *** add line here ***
        if (freq > Fmax) freq = Fmax;
        if (freq < Fmin) freq = Fmin;

        // calculate the phase (modulo 2*pi)
        // *** add line here ***
        phase= 2* 3.141592654* idtmod(freq, 0, 1, -0.5);

        // generate the output
        sine = sin(phase);

        @(cross(sine,0))
        ;

        if (sine > 0)
            // *** add line here ***
            VOUT_i= 1;
        else
            // *** add line here ***
            VOUT_i= 0;

        V(OUT) <+ transition(VOUT_i*VDD,td,trise,tfall);

        // bound the time step
        // *** add line here ***
        $bound_step(0.1/freq);

    end

endmodule
```

Figure 8: VCO VerilogA Model

DIVIDER:

```
// VerilogA for Analog_Course, pll_Divider, veriloga

`include "constants.vams"
`include "disciplines.vams"

module serdes_iti_lab7_pll_DIVIDER(CLKIN, CLKOUT, AVDD, AVSS);
inout CLKIN, CLKOUT, AVDD, AVSS;
electrical CLKIN, CLKOUT, AVDD, AVSS;

    parameter real vh=1.2;          // output voltage in high state
    parameter real vl=0;            // output voltage in low state
    parameter real vth=0.6;        // threshold voltage at input
    parameter integer ratio=4 from [2:inf]; // divider ratio
    parameter real tt=10p from (0:inf); // transition time of output signal
    parameter real td=0 from [0:inf]; // average delay from input to output

    // *** add line here ***
    real out_value=0;
    integer count=0;

    analog begin

        @(cross(V(CLKIN) - vth, 1)) begin
            if (count==floor(ratio/2))
                out_value = vh;
            // *** add line here ***
            if (count==ratio) begin
                out_value = vl;
                // *** add line here ***
                count= 0;
            end
            count = count + 1;
        end

        V(CLKOUT) <+ transition(out_value, td, tt);

    end

endmodule
```

Figure 9: Divider VerilogA Model



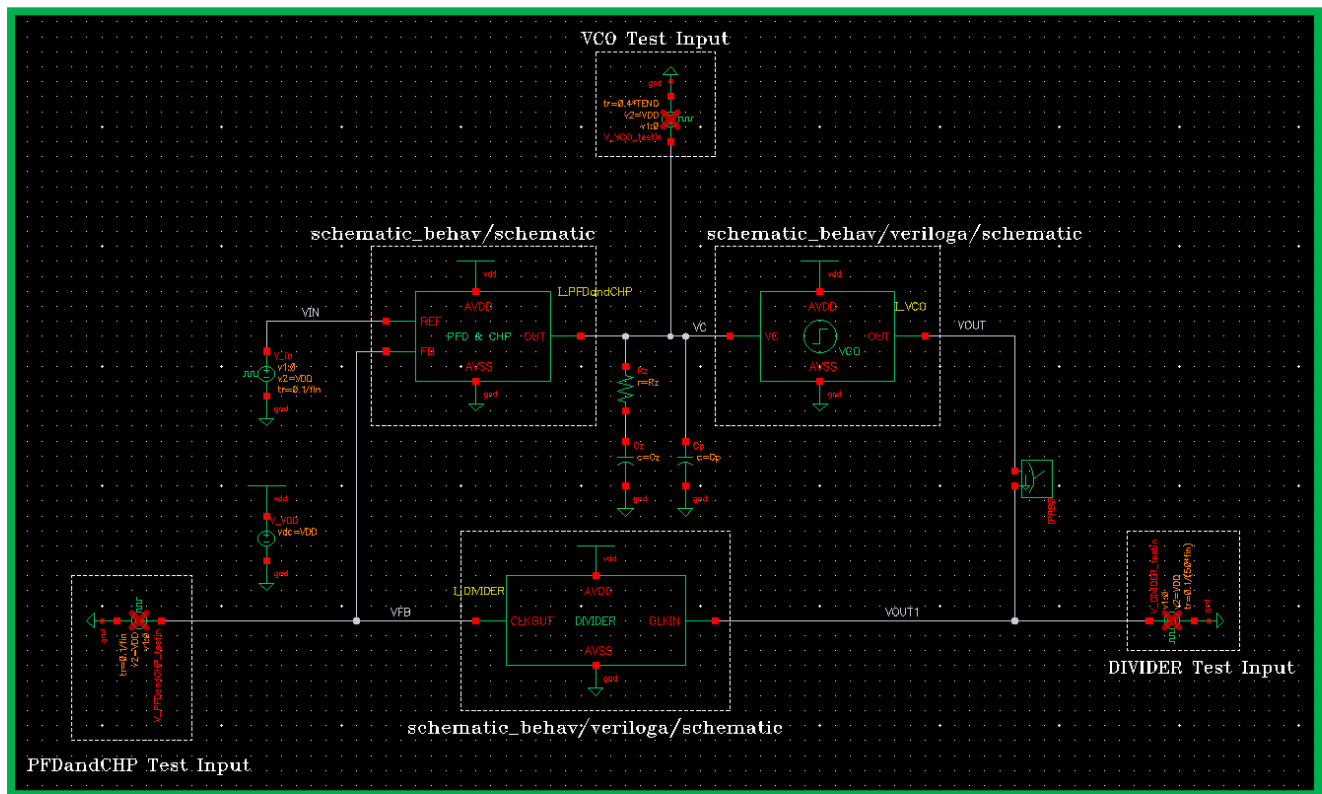
Figure 10: VerilogA Parameters

$$t_r = \frac{0.1}{f_{max}}, \quad VCO \text{ frequency} = [3G:7G]$$

➔ Hint: Use the calculated ICP, Rz, Cz, Cp, N from Part 1.

2. Sub-Block Testing

➔ Hint: You can work in the main TB, and press “shift-delete” on the blocks you don’t want to include in your simulation.



2.1. Create a testbench to test PFD. Copy the input (reference) pulse source, add a $\frac{1}{4}$ cycle positive delay, then put it in the FB input. Run transient analysis for 5 cycles.

➔ Hint: Put $TEND=5/fin$.

2.1.a. Plot VIN, VFB, UP, UPB, DN & DNB in the same graph.

2.1.b. Modify the FB input to have a negative delay. Repeat 2.1.a.

2.2. Create a testbench to test CHP. Make the same testbench in 2.1, and add CHP block & (R_z , C_z , C_p) after CHP. Run transient analysis for 50 cycles.

2.2.a. Plot UP, DN & VC in the same graph.

2.2.b. Modify the FB input to have a negative delay. Repeat 2.2.a.

2.3. Create a testbench to test VCO. Create a ramping up and down source as a “VC” input to the VCO. Run transient analysis for 1 ramp-up & down. Plot freq(VOUT) vs VC.

Voltage 1	0 V
Voltage 2	VDD V
Period	TEND s
Delay time	0.1*TEND s
Rise time	0.4*TEND s
Fall time	0.4*TEND s
Pulse width	0.1*TEND s

➔ Hint#1: Use these pulse input parameters: $TEND(3us)$, run from 0:

➔ Hint#2: You may use “waveVsWave()” function.

- 2.4. Create a testbench to test DIVIDER. Make its input a 5GHz clock, run transient for 200 cycles and plot input & output signals. Comment.

3. Transient Simulation

- 3.1. Run transient simulation to VAR("TEND"), and put TEND=3u. Plot VC. Comment.
- 3.2. Plot freq(VIN) & freq(VFB) in the same graph. Is there a frequency error between VIN & VFB? Why?
- 3.3. Plot freq(VOUT). Is it similar to any plotted waveform before? Why?

Part 3: PLL Circuit Implementation (Bonus)

1. TestBench Setup

- Create new schematic views for “PFD”, “CHP”, “VCO”, “DIVIDER”.

CHP:

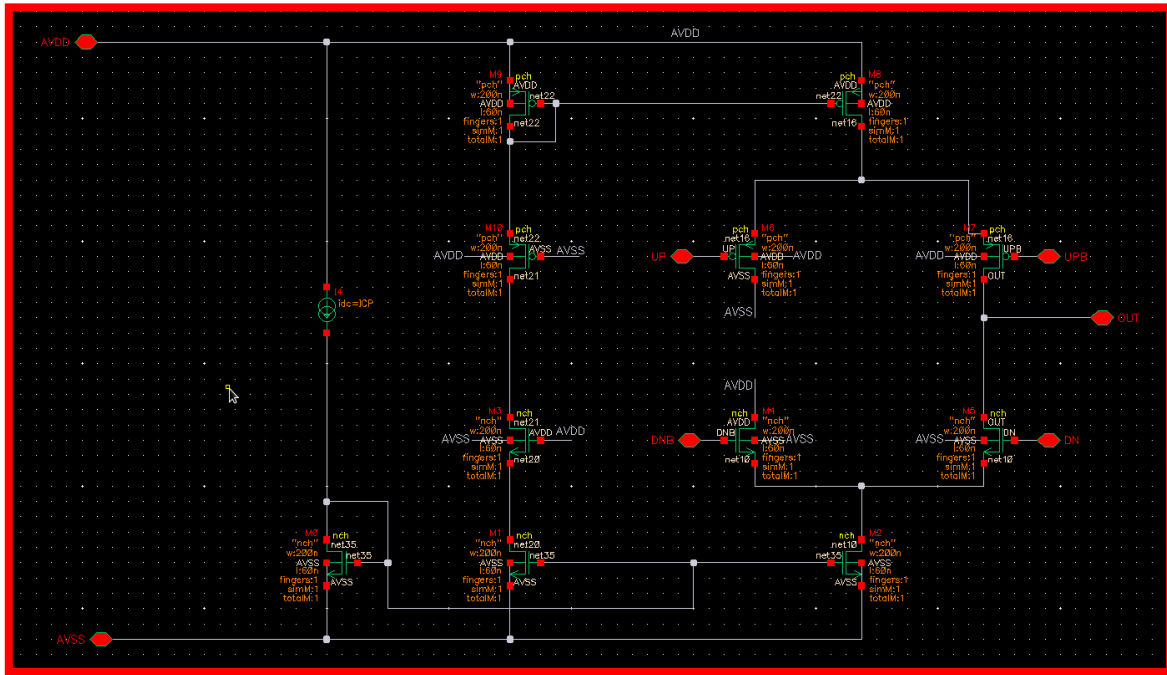


Figure 11: CHP Circuit Implementation

PFD:

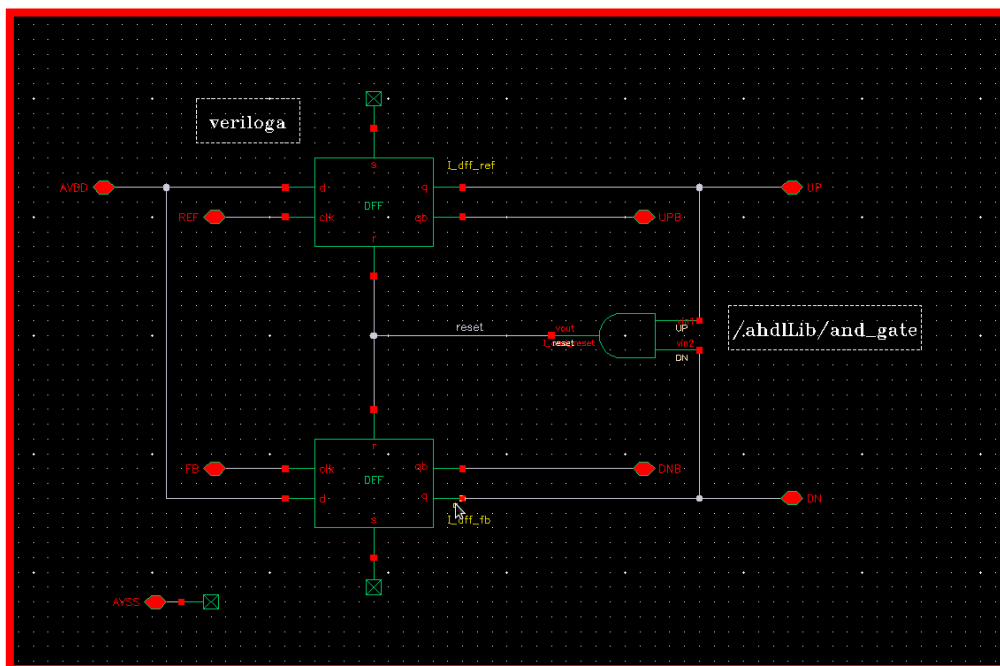


Figure 12: PFD schematic using DFF verilog model

```

// Simple edge-triggered D flip-flop
// Version 1c, 17 July 03
// Ken Kundert
// Downloaded from The Designer's Guide (www.designers-guide.org).
// Post any questions to www.designers-guide.org/Forum

`include "disciplines.vams"

// D-flip flop with clear or reset

module dff2 (q, qb, clk, d, s, r);

output q; voltage q; // Q output
output qb; voltage qb; // Q bar output
input clk; voltage clk; // Clock input (edge triggered)
input d; voltage d; // D input
input s; voltage s; // Set input (immediately forces Q output high) (active high)
input r; voltage r; // Reset input (immediately forces Q output low) (active high)
parameter real td = 0 from [0:inf); // delay from clock to q
parameter real tt = 0 from [0:inf); // transition time of output signals
parameter real vh = 1; // output voltage in high state
parameter real vl = -1; // output voltage in low state
parameter real vth = (vh + vl)/2; // threshold voltage at inputs
parameter integer dir = +1 from [-1:+1] exclude 0;
// if dir=+1, rising clock edge triggers flip flop
// if dir=-1, falling clock edge triggers flip flop

real out, outb;

analog begin
    @(initial_step) begin
        out = vl;
        outb = vh;
    end
    @(cross(V(clk) - vth, dir)) begin
        if (V(d) > vth) begin
            out = vh;
            outb = vl;
        end else begin
            out = vl;
            outb = vh;
        end
    end
    if (V(s) > vth) begin
        out = vh;
        outb = vl;
    end
    if (V(r) > vth) begin
        out = vl;
        outb = vh;
    end
    end
    V(q) <+ transition( out, td, tt );
    V(qb) <+ transition( outb, td, tt );
end
endmodule

```

Figure 13: DFF with Reset VerilogA Model

VCO:

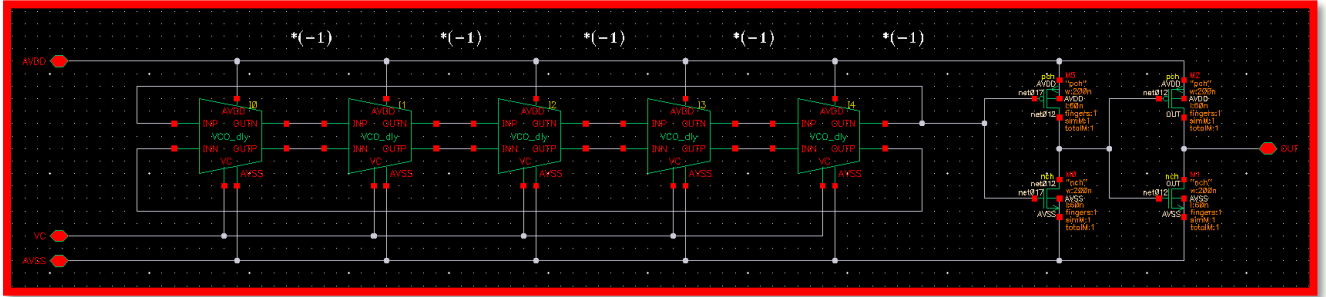


Figure 14: VCO circuit schematic using delay cells

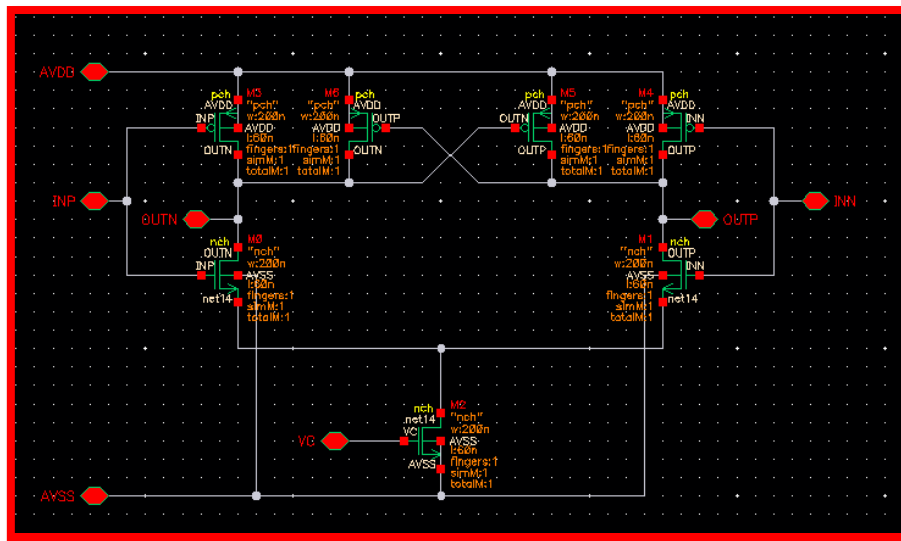


Figure 15: Delay Cell circuit schematic

- 1.1. Knowing that $f_{VCO} = 1/(2N * t_{dly})$, how to decrease VCO output frequency range using the same unit delay circuit?
- 1.2. Design a divide-by-50 frequency divider, using dff's & nand gates from ahdlLib.

→ Hint#1: You can use the following divide-by-5 & divide-by-2 circuits.

➔ Hint#2: Make sure to add a non-zero value of trise/tfall/logic_high/vtrans/vtrans_clk for every logic cell from ahdlLib (vtrans is the voltage threshold between high & low).

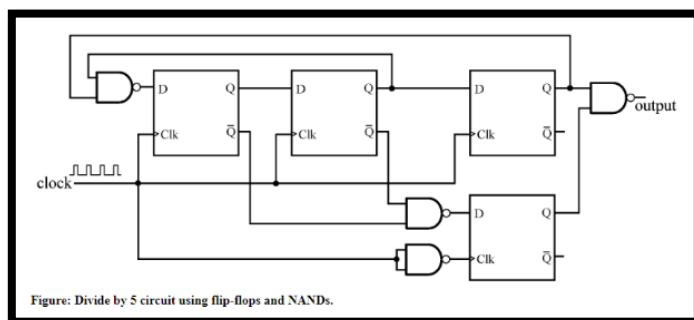
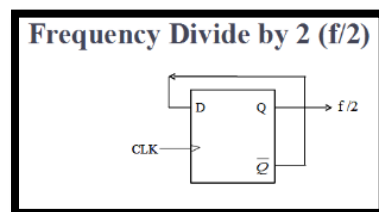


Figure: Divide by 5 circuit using flip-flops and NANDs.



Frequency Divide by 2 ($f/2$)

Figure 16: DIVIDER sub-circuits

2. Sub-Block Testing

- 2.1. Open the PFD TB you created in Part 2. Make a $\frac{1}{4}$ cycle positive delay between reference and FB. Run transient analysis for 5 cycles.
 - 2.1.a. Plot VIN, VFB, UP, UPB, DN & DNB in the same graph.
 - 2.1.b. Modify the FB input to have a negative delay. Repeat 2.1.a. Comment on the results.
 - 2.1.c. Now, add a 1-cycle delay in VIN & $\frac{3}{4}$ cycle delay in VFB. Repeat 2.1.a. Comment.
- 2.2. Open the CHP TB you created in Part 2. Make the same testbench in 2.1. Run transient analysis for 50 cycles.
 - 2.2.a. Plot UP, DN & VC in the same graph.
 - 2.2.b. Add a 1-cycle delay in VIN & $\frac{3}{4}$ cycle delay in VFB. Repeat 2.2.a.
- 2.3. Open the VCO TB you created in Part2. Run transient analysis for 1 ramp-up & down. Plot freq(VOUT) vs VC. Comment on simulation time of this run.

Voltage 1	0 V
Voltage 2	VDD V
Period	TEND s
Delay time	0.1*TEND s
Rise time	0.4*TEND s
Fall time	0.4*TEND s
Pulse width	0.1*TEND s

- ➔ Hint#1: Use these pulse input parameters: TEND(3us), run from 0:
- ➔ Hint#2: You may use “waveVsWave()” function.

- 2.4. Open the DIVIDER TB you created in Part2. Run transient for 200 5GHz-cycles and plot input & output signals. Comment.

3. Transient Simulation

- 3.1. Run transient simulation to VAR(“TEND”), and put TEND=3u. Plot VC.
- 3.2. Plot freq(VIN) & freq(VFB) in the same graph.
- 3.3. Plot freq(VOUT).
- 3.4. Compare these waveforms with ones from Part 2. Is there a difference? Why?

References

- ASU ECE621 Course, Dr. Sameh Ibrahim.
- Analog Mixed Signals Course, Dr. Hesham Omran.
- designers-guide.org